Boolean Algebra

Discrete Math, Fall 2025

Konstantin Chukharev

Boolean Algebra

“Мы почитаем всех нулями,
А единицами — себя.”

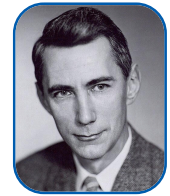
— А.С. Пушкин, «Евгений Онегин»

Gottfried Wilhelm Leibniz

George Boole

Augustus De Morgan

Charles Sanders Peirce



Claude Shannon

From Algebra to Digital Circuits

Historical note: George Boole's groundbreaking work *The Laws of Thought* (1854) created an algebraic system for logic a full century before the first electronic computers. He showed that logical reasoning could be captured by equations, replacing Aristotle's verbal arguments with mathematical symbols. This insight became the theoretical foundation for the digital age.

Key idea: Just as ordinary algebra works with numbers and operations like $+$ and $\times$, Boolean algebra works with **truth values** (0 and 1, or `false` and `true`) and operations like AND, OR, and NOT.

Historical milestones:

- **1703:** Leibniz describes binary arithmetic
- **1854:** Boole publishes algebraic logic
- **1937:** Shannon applies it to circuits
- **1940s:** First digital computers

Modern applications:

- Processor design (CPUs, GPUs)
- Formal verification (correctness proofs)
- Database optimization (queries)
- AI reasoning (SAT, planning)

Boolean Values: 0 and 1

In Boolean algebra, we work with exactly two values:

- **0** (`false`, off, low voltage)
- **1** (`true`, on, high voltage)

Example: In different contexts:

- **Mathematics:** "Is $x > 5$?" $\longrightarrow$ `true` (1) or `false` (0)
- **Programming:** `if (x && y)` $\longrightarrow$ evaluates to `true` or `false`
- **Circuits:** Wire voltage $\longrightarrow$ high (1) or low (0)
- **Sets:** Is element in set? $\longrightarrow$ yes (1) or no (0)

Why this matters: Modern computers represent all information using binary (0s and 1s).

Boolean algebra is the mathematics that makes digital computation possible.

What You Will Learn

In this lecture, you'll master Boolean algebra from foundations to applications:

1. Build Boolean expressions and evaluate them using **truth tables**
2. Prove identities using the **fundamental laws** (commutativity, De Morgan's laws, distributivity)
3. **Synthesize any Boolean function** from specifications using CNF, DNF, and Karnaugh maps
4. **Simplify complex expressions** through algebraic manipulation and graphical methods
5. Determine which operation sets are **functionally complete** using Post's criterion
6. **Design real digital circuits** with logic gates, flip-flops, and stateful elements
7. Apply Boolean algebra to **modern CS problems**: binary decision diagrams, SAT solving, and formal verification

Our goal: You'll gain both theoretical understanding (algebraic structure, formal proofs) and practical skills (circuit design, expression minimization). By the end, you'll see how the same mathematical framework powers everything from smartphone processors to AI planning algorithms.

Variables and Expressions

The Language of Boolean Logic

Just like natural languages have words and grammar rules, Boolean algebra has its own building blocks and composition rules.

Natural Language:

- Words: “cat”, “dog”
- Connectors: “and”, “or”, “not”
- Sentences: “The cat is black and the dog is not white”

Boolean Algebra:

- Variables: x, y
- Operations: $\wedge$, $\vee$, $\neg$
- Expressions: $x \wedge \neg y$

Goal: Learn the “grammar” of Boolean logic — how to write and read Boolean expressions that represent logical relationships.

Boolean Variables: The Atoms

Definition 1: A *Boolean variable* is a variable that can take only one of two values: 0 (false) or 1 (true).

Think of Boolean variables as yes/no questions:

Example: Real-world Boolean variables (in programming):

- `is_logged_in` — Is the user logged in? (yes/no)
- `has_permission` — Does user have permission? (yes/no)
- `sensor_triggered` — Did the sensor activate? (yes/no)
- $x > 5$ — Is x greater than 5? (true/false)

Why binary? Digital circuits use voltage levels (high/low), making binary the natural choice for computation. Every piece of data in your computer is ultimately represented as 0s and 1s.

Boolean Operations: Connecting Ideas

We combine Boolean variables using operations (connectives) to express complex logic:

Definition 2 (Basic Operations):

- **NOT** ($\neg x$ or $\overline{x}$) — *reverses* the value
- **AND** ($x \wedge y$ or $x \cdot y$) — true only if *both* are true
- **OR** ($x \vee y$ or $x + y$) — true if *at least one* is true

Example: Access control logic:

- `is_admin OR has_permission` — User needs admin status OR explicit permission
- `is_logged_in AND NOT is_banned` — User must be logged in AND not banned
- `NOT (sensor1 AND sensor2)` — Not both sensors triggered simultaneously

Watch out: "OR" in Boolean logic is *inclusive* (can be both), unlike everyday speech where "coffee or tea" usually means "pick one."

Derived Operations: Shortcuts

More complex operations built from basic ones:

Definition 3 (Derived Operations):

- **XOR** ($x \oplus y$): true if *exactly one* is true
  - Formula: $(x \wedge \neg y) \vee (\neg x \wedge y)$
  - Example: "Dessert or coffee" (pick one, not both)
- **Implication** ($x \rightarrow y$): "if x then y"
  - Formula: $\neg x \vee y$
  - Example: "If it rains, bring umbrella"
- **Equivalence** ($x \Longleftrightarrow y$): true when values *match*
  - Formula: $(x \rightarrow y) \wedge (y \rightarrow x)$
  - Example: "Light is on if and only if switch is up"

Building Boolean Expressions

Definition 4: A *Boolean expression* is built recursively from variables and operations:

1. Variables (x, y, z) and constants ($0, 1$) are expressions
2. If f and g are expressions, so are: $\neg f$, $f \wedge g$, $f \vee g$, $f \oplus g$, $f \to g$, $f \iff g$

Example: Progressive complexity:

1. x — atomic variable
2. $\neg x$ — negation
3. $x \wedge y$ — two variables connected
4. $(x \wedge y) \vee z$ — nested expression
5. $\neg(x \vee (y \wedge z))$ — deeply nested
6. $(x \to y) \wedge (y \to z)$ — transitivity pattern

Key idea: Operations work on *any* expressions, not just variables. This compositionality lets us build arbitrarily complex formulas.

Truth Tables

Definition 5: A *truth table* is a complete list of all possible input combinations for a Boolean expression and their corresponding output values.

Note: For n Boolean variables, there are 2^n possible input combinations.

Example: Truth table for AND operation ($x \wedge y$):

x	y	$x \wedge y$
0	0	0
0	1	0
1	0	0
1	1	1

Truth Tables for Compound Expressions

We can build truth tables for complex expressions step by step.

Example: Truth table for $(x \wedge y) \vee (\neg x \wedge z)$:

x	y	z	$x \wedge y$	$\neg x \wedge z$	$(x \wedge y) \vee (\neg x \wedge z)$
0	0	0	0	0	0
0	0	1	0	1	1
0	1	0	0	0	0
0	1	1	0	1	1
1	0	0	0	0	0
1	0	1	0	0	0
1	1	0	1	0	1
1	1	1	1	0	1

Method: Build intermediate columns for subexpressions, then combine them for the final result.

Tautologies, Contradictions, Contingencies

Definition 6:

- A *tautology* is a Boolean expression that is always true (all 1s in output column)
- A *contradiction* is a Boolean expression that is always false (all 0s in output column)
- A *contingency* is a Boolean expression that is sometimes true and sometimes false

Example:

- **Tautology:** $x \vee \neg x$ (law of excluded middle)
- **Contradiction:** $x \wedge \neg x$ (law of non-contradiction)
- **Contingency:** $x \wedge y$ (depends on values of x and y)

Warning: In Boolean algebra, we typically care about expressions that are contingencies—those that represent actual functions. Tautologies and contradictions are constant functions.

Logical Equivalence

Definition 7: Two Boolean expressions f and g are *logically equivalent* (written $f \equiv g$) if they have identical truth tables—they produce the same output for every possible input combination.

Example: Show that $\neg(x \wedge y) \equiv \neg x \vee \neg y$ (De Morgan's law):

x	y	$\neg(x \wedge y)$	$\neg x \vee \neg y$	**Equal?**
0	0	1	1	✓
0	1	1	1	✓
1	0	1	1	✓
1	1	0	0	✓

Since all rows match, $\neg(x \wedge y) \equiv \neg x \vee \neg y$. ■

Evaluating Boolean Expressions

Given values for all variables, we can evaluate any Boolean expression.

Example: Evaluate $f(x, y, z) = (x \vee y) \wedge (\neg x \vee z)$ when $x = 1, y = 0, z = 1$:

$$\begin{aligned} f(1, 0, 1) &= (1 \vee 0) \wedge (\neg 1 \vee 1) \\ &= 1 \wedge (0 \vee 1) \\ &= 1 \wedge 1 \\ &= 1 \end{aligned}$$

Practice skill: Being able to quickly evaluate Boolean expressions is essential for debugging circuits and verifying logical designs.

Simplifying Boolean Expressions

Simple algebraic manipulation can simplify expressions:

Example: Simplify $f(x, y) = (x \wedge y) \vee (x \wedge \neg y)$:

$$\begin{aligned} f(x, y) &= (x \wedge y) \vee (x \wedge \neg y) \\ &= x \wedge (y \vee \neg y) && \text{(distributivity)} \\ &= x \wedge 1 && \text{(excluded middle)} \\ &= x && \text{(identity)} \end{aligned}$$

Example: Simplify $g(x, y, z) = (x \vee y) \wedge (x \vee \neg y)$:

$$\begin{aligned} g(x, y, z) &= (x \vee y) \wedge (x \vee \neg y) \\ &= x \vee (y \wedge \neg y) && \text{(distributivity)} \\ &= x \vee 0 && \text{(contradiction)} \\ &= x && \text{(identity)} \end{aligned}$$

Boolean Algebra Structure

Definition and Basic Properties

Definition 8: A *Boolean algebra* is a bounded distributive lattice $(B, \vee, \wedge, \bot, \top)$ with complement $(\cdot)' : B \to B$ such that $x \vee x' = \top$ and $x \wedge x' = \bot$.

Example: $(\mathcal{P}(A), \cup, \cap, \emptyset, A)$ with $X' = A \setminus X$ is a Boolean algebra.

Example (Digital Circuit Design): Consider 3-bit binary values as Boolean algebra:

- Elements: $\{000, 001, 010, 011, 100, 101, 110, 111\}$
- Order: Bitwise comparison ($001 \leq 011$ since $0 \leq 0$, $0 \leq 1$, $1 \leq 1$)
- Join: Bitwise OR ($010 \vee 101 = 111$)
- Meet: Bitwise AND ($110 \wedge 101 = 100$)
- Complement: Bitwise NOT ($001' = 110$)

This directly corresponds to logic gates: OR, AND, NOT gates in computer processors.

Note: Logical reading: "join" $\mapsto \vee$, "meet" $\mapsto \wedge$, "complement" $\mapsto \neg$.

Example: Database Query Lattice

Example: A database has tables `Students`, `Courses`, `Enrollments`.

- Let Q_1 = "Computer Science students"
- Let Q_2 = "Students in Math courses"
- Let Q_3 = "Graduate students"

Consider queries as lattice elements ordered by result size (specificity).

Lattice Operations:

- $Q_1 \vee Q_2$ = "Students in CS OR Math courses" (larger result set)
- $Q_1 \wedge Q_2$ = "CS students taking Math courses" (smaller result set)
- $Q_1 \wedge Q_3$ = "Graduate CS students" (most specific)

Why this matters: Query optimizers use this structure to:

1. Find equivalent but more efficient queries.
2. Cache common subqueries.
3. Predict result set sizes for cost estimation.

Complement is Unique

Theorem 1: Complements are unique in a Boolean algebra.

Proof: Suppose for some element a we have *two* complements x and y.

$$
\begin{aligned}
x &= x \wedge \top && \top \text{ is the identity for } \wedge \\
&= x \wedge (a \vee y) && \text{by definition of complement: } \top = a \vee y \\
&= (x \wedge a) \vee (x \wedge y) && \wedge \text{ distributes over } \vee \\
&= \bot \vee (x \wedge y) && \text{by definition of complement: } \ x \wedge a = \bot \\
&= (a \wedge y) \vee (x \wedge y) && \text{by definition of complement: } \ \bot = a \wedge y \\
&= (a \vee x) \wedge y && \wedge \text{ distributes over } \vee \\
&= \top \wedge y && \text{by definition of complement: } \ a \vee x = \top \\
&= y && \top \text{ is the identity for } \wedge
\end{aligned}
$$

That is, $x = y$. $\square$

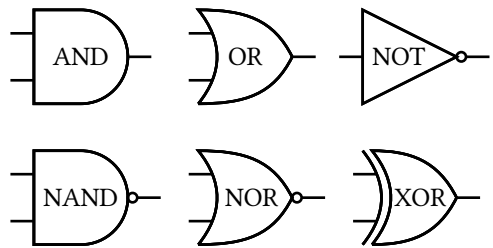
De Morgan's Laws

Theorem 2 (De Morgan): $(x \vee y)' = x' \wedge y'$ and $(x \wedge y)' = x' \vee y'$ in any Boolean algebra.

Digital Logic Circuits

Definition 9: A *logic gate* is a physical device that implements a Boolean function, taking binary inputs and producing a binary output.

Gate	Formula	Description
AND	$A \wedge B$	Outputs 1 only when both inputs are 1
OR	$A \vee B$	Outputs 1 when at least one input is 1
NOT	$\neg A$	Outputs the opposite of the input
NAND	$\neg(A \wedge B)$	Outputs 0 only when both inputs are 1
NOR	$\neg(A \vee B)$	Outputs 0 when at least one input is 1
XOR	$A \oplus B$	Outputs 1 when inputs differ
XNOR	$A \equiv B$	Outputs 1 when inputs are the same



Note: NAND and NOR gates are *universal* — any Boolean function can be implemented using only NAND gates (or only NOR gates). For example, to implement AND using NAND:

$$A \wedge B = \neg\neg(A \wedge B) = \neg(A \overline{\wedge} B) = (A \overline{\wedge} B) \overline{\wedge} (A \overline{\wedge} B)$$

Combinational Logic

Definition 10: A *combinational circuit* is a circuit where the output depends only on the current input values, without any memory or state.

Example (Half Adder): Adds two single bits:

- Sum: $S = A \oplus B$
- Carry: $C = A \wedge B$

Example (Full Adder): Adds two bits plus a carry-in:

- Sum: $S = A \oplus B \oplus C_{\text{in}}$
- Carry-out: $C_{\text{out}} = (A \wedge B) \vee (C_{\text{in}} \wedge (A \oplus B))$

Sequential Logic and Memory

Definition 11: A *sequential circuit* is a circuit where the output depends on both current inputs and previous state (memory).

Example (Flip-Flops):

- **SR Latch**: Set-Reset memory element.
- **D Flip-Flop**: Data storage triggered by clock edge.
- **JK Flip-Flop**: Eliminates forbidden state of SR latch.
- **T Flip-Flop**: Toggle flip-flop for counters.

Normal Forms

Definition 12: A *literal* is a Boolean variable or its negation (*e.g.*, x, $\neg x$).

Definition 13 (DNF): A Boolean formula is in *disjunctive normal form (DNF)* if it is a disjunction (OR) of *terms* — conjunctions (AND) of literals.

Example: $f(x, y, z) = \underbrace{(x \wedge y \wedge \neg z)}_{\text{term}} \vee \underbrace{(\neg x \wedge z)}_{\text{term}} \vee \underbrace{(\neg y \wedge \neg z)}_{\text{term}} \vee \underbrace{x}_{\text{term}}$

Definition 14 (CNF): A Boolean formula is in *conjunctive normal form (CNF)* if it is a conjunction (AND) of *clauses* — disjunctions (OR) of literals.

Example: $f(x, y, z) = \underbrace{(x \vee y \vee \neg z)}_{\text{clause}} \wedge \underbrace{(\neg x \vee z)}_{\text{clause}} \wedge \underbrace{(\neg y \vee \neg z)}_{\text{clause}} \wedge \underbrace{x}_{\text{clause}}$

Minterms and Maxterms

Definition 15 (Minterm and Maxterm):

- A *minterm* is a product (AND) of literals where each variable appears exactly once.
- A *maxterm* is a sum (OR) of literals where each variable appears exactly once.

Note: A minterm (maxterm) is a function that evaluates to 1 (0, respectively) for exactly one combination of variable values.

Example: $f(x, y, z) = x\overline{y}z$ is a minterm, and $g(x, y, z) = x + \overline{y} + z$ is a maxterm for variables x, y, z.

- $f(x, y, z) = 1$ only on input 101, *i.e.*, $x = 1$, $y = 0$, $z = 1$, corresponding to the minterm $x\overline{y}z$.
- $g(x, y, z) = 0$ only on input 010, *i.e.*, $x = 0$, $y = 1$, $z = 0$, correspending to the maxterm $\overline{x} + y + \overline{z}$.

Canonical Forms

Definition 16 (SoP): Every Boolean function can be *uniquely* expressed as a *sum of minterms* (SoP, Sum of Products) corresponding to rows where the function evaluates to 1.

Definition 17 (PoS): Every Boolean function can be *uniquely* expressed as a *product of maxterms* (PoS, Product of Sums) corresponding to rows where the function evaluates to 0.

Karnaugh Maps

Definition 18: A *Karnaugh map* (K-map) is a graphical method for simplifying Boolean expressions by visually identifying adjacent minterms that can be combined.

AB \ CD	**00**	**01**	**11**	**10**
00	**0**	**1**	**3**	**2**
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

Zhegalkin Polynomials

Definition 19: A *Zhegalkin polynomial* is a representation of a Boolean function as a polynomial over $\mathrm{GF}(2)$ using XOR ($\oplus$) and AND ($\wedge$, often omitted) operations.

Theorem 3: Every Boolean function has a unique representation as a Zhegalkin polynomial:

$$f(x_1, \ldots, x_n) = \bigoplus_{S \subseteq \{1,\ldots,n\}} \left(a_S \prod_{i \in S} x_i \right)$$

where $a_S \in \{0, 1\}$ and $\oplus$ denotes XOR.

Example: $f(x, y) = x \vee y = x \oplus y \oplus xy$

Binary Decision Diagrams (BDDs)

Definition 20 (BDD): A *binary decision diagram (BDD)* is a directed acyclic graph representing a Boolean function, where each non-terminal node represents a variable test and edges represent variable assignments.

Definition 21 (ROBDD): A *reduced* ordered binary decision diagram (ROBDD) is an ordered BDD with a fixed variable ordering where:

- No variable appears more than once on any path
- No two nodes have identical low and high successors
- No node has identical low and high successors

Theorem 4: Every Boolean function has a unique reduced ordered binary decision diagram (ROBDD) representation for a given variable ordering.

TODO

- ...